

COMSATS UNIVERSITY ISLAMABAD , ATTOCK CAMPUS

Information Security

Name:

M. Haider Iqbal

Reg No:

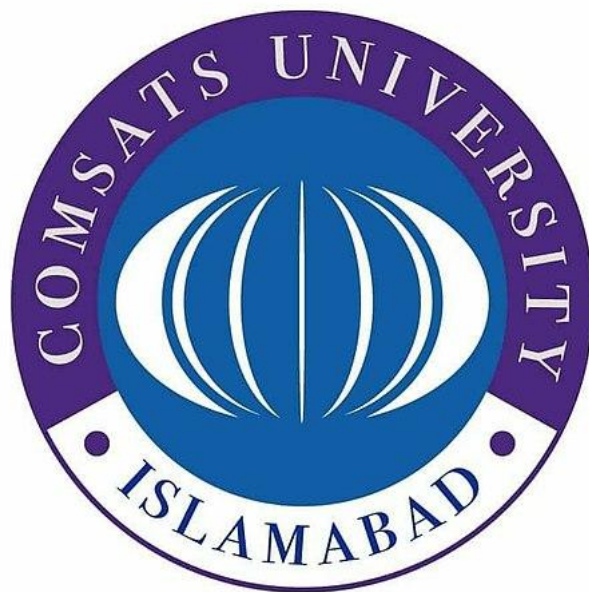
SP24-BSE-039

Submitted to:

Miss Ambreen Gul

Assignment No:

04



Task 1:

```
def gcd(a, b):  
    while b:  
        a, b = b, a % b  
    return a
```

```
def extended_gcd(a, b):  
    if b == 0:  
        return (a, 1, 0)  
    g, x1, y1 = extended_gcd(b, a % b)  
    x = y1  
    y = x1 - (a // b) * y1  
    return (g, x, y)
```

```
def modinv(a, m):  
    g, x, y = extended_gcd(a, m)  
    if g != 1:  
        raise ValueError("Modular inverse does not exist")  
    return x % m
```

```
def generate_keys(p=211, q=223, e=17):  
    # Using larger primes so messages fit inside modulus  
    n = p * q  
    phi = (p - 1) * (q - 1)
```

```
    if gcd(e, phi) != 1:  
        raise ValueError("e and phi(n) must be coprime")
```

```
    d = modinv(e, phi)
```

```
    return (n, e), (n, d)
```

```
# Convert text to integer and back  
def text_to_int(msg: str) -> int:  
    return int.from_bytes(msg.encode('utf-8'), 'big')
```

```
def int_to_text(m: int) -> str:  
    try:  
        return m.to_bytes((m.bit_length() + 7) // 8, 'big').decode('utf-8')
```

```
except:
    return "<decode error>"
```

```
def encrypt_integer(m_int: int, pubkey: tuple) -> int:
    n, e = pubkey
    return pow(m_int, e, n)
```

```
def decrypt_integer(c_int: int, privkey: tuple) -> int:
    n, d = privkey
    return pow(c_int, d, n)
```

```
def encrypt_message(msg: str, pubkey: tuple) -> int:
    m_int = text_to_int(msg)
    if m_int >= pubkey[0]:
        raise ValueError("Message too large for RSA modulus. Use larger primes.")
    return encrypt_integer(m_int, pubkey)
```

```
def decrypt_message(cipher_int: int, privkey: tuple) -> str:
    m_int = decrypt_integer(cipher_int, privkey)
    return int_to_text(m_int)
```

```
# -----
# EXAMPLE RUN
# -----
```

```
if __name__ == '__main__':
    public, private = generate_keys() # uses improved primes
    name = "Alice" # now works without error
```

```
print("Public key:", public)
print("Private key:", private)
```

```
cipher = encrypt_message(name, public)
print("Encrypted (int):", cipher)
```

```
plain = decrypt_message(cipher, private)
print("Decrypted text:", plain)
```

Task 2:

```
from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP
from Crypto.Random import get_random_bytes
```

```
def generate_rsa_2048():
    key = RSA.generate(2048)
    private_key_pem = key.export_key()
    public_key_pem = key.publickey().export_key()
    return private_key_pem, public_key_pem, key
```

```
def encrypt_message(message: bytes, public_key):
    cipher = PKCS1_OAEP.new(public_key)
    return cipher.encrypt(message)
```

```
def decrypt_message(ciphertext: bytes, private_key):
    cipher = PKCS1_OAEP.new(private_key)
    return cipher.decrypt(ciphertext)
```

```
if __name__ == '__main__':
    message = b"Hello from COMSATS student"
    priv_pem, pub_pem, keyobj = generate_rsa_2048()
    print("Private key (PEM, trimmed):\n", priv_pem[:200], b"... \n")
    print("Public key (PEM, trimmed):\n", pub_pem[:200], b"... \n")
    pub = keyobj.publickey()
    ciphertext = encrypt_message(message, pub)
    print("Ciphertext (hex):", ciphertext.hex())
    plaintext = decrypt_message(ciphertext, keyobj)
    print("Recovered plaintext:", plaintext.decode('utf-8'))
```

Task 3:

```
from Crypto.PublicKey import RSA
from Crypto.Signature import pkcs1_15
from Crypto.Hash import SHA256
```

```
def generate_keypair(bits=2048):
    key = RSA.generate(bits)
    return key, key.publickey()
```

```
def sign_message(message: bytes, private_key):
    h = SHA256.new(message)
    signature = pkcs1_15.new(private_key).sign(h)
    return signature
```

```
def verify_signature(message: bytes, signature: bytes, public_key):
    h = SHA256.new(message)
    try:
        pkcs1_15.new(public_key).verify(h, signature)
        return True
    except (ValueError, TypeError):
        return False
```

```
if __name__ == '__main__':
    msg = b"This message will be signed."
    priv, pub = generate_keypair()
    sig = sign_message(msg, priv)
    print("Signature (hex):", sig.hex())
    ok = verify_signature(msg, sig, pub)
    print("Verification (original message):", ok)
    # modify the message
    msg2 = b"This message will be signed?"
    ok2 = verify_signature(msg2, sig, pub)
    print("Verification (modified message):", ok2)
```

Task 1 Output:

Public key: (47053, 17)

Private key: (47053, 38393)

Task 2 Output:

Private key (PEM, trimmed):

```
b'-----BEGIN RSA PRIVATE KEY-----
\nMIIEowIBAAKCAQEA2B2+y1i9P0jemZtfwQbjU3CBqBs+ZXi7V+SQp1Cq7
xw75A4V\nj30FRijgBNG1lur/bMaGCfnMsV1onNeBcHkEk+JOV7H/5FjZF4
D/IA+hAn1Hvcj\nfTT7vs5kzfyFhG0XG9Nq/HBhbmEboXdqtGNjYP' b'...\n'
```

Public key (PEM, trimmed):

```
b'-----BEGIN PUBLIC KEY-----
\nMIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEA2B2+y1i9P0je
mZtfwQbj\nU3CBqBs+ZXi7V+SQp1Cq7xw75A4Vj30FRijgBNG1lur/bMaG
CfnMsV1onNeBcHk\nEk+JOV7H/5FjZF4D/IA+hAn1HvcjTT7vs5kzfyFhG0'
b'...\n'
```

Ciphertext (hex):

```
5979a19911d785948e320bee1515bde3c0bed388fa1a8185e20cdec4ac4
29ac6f26880476f00e891936b3b64d13ee581d7ed50739443b76f0c9e64
dc187fcbf5cf79cdbbbc8424aa49ffd0d480b3629e22f1420543c064c354f8
73f9dc9c8e99118ba54e69004f216aca02881827397dd0b7f10f0d2705b2
167de10942eaab8dff229fb744a8c6faafdf72eaf07adf541f56d71970f362
b918a397132d7183b699530d6a42471cffe9b3ffb09c30139ff6ad722c248
a0e4622e492cb88bc8f2dd0cc70e9c2264c6d004f65a4b9b3efda4125623f
ee03caad12cc141e1b7b256410bb352de5c67eed396d335611e5479dee5
0e3f725a3209c4ac25b7b4a9c8230
```

Recovered plaintext: Hello from COMSATS student

Task 3 Output:

Signature (hex):

```
60a9056266ded8c1fda2c6dda254ab9b988a487e78fe538d749651981c6a
a4bf8c7c4e3462c701c7106570ef5363b8d4f4acdf9bb244245729f952477
```

1a57122e08f40783ef14ca7cc77691ff701d9222d34d616293ae1d3806bfa
7186edb85982f8e4b97bc28864af4071b46b8bb6826793ff21b3f1483ddb
ba76d9d7b6cdfa90ad27c6325538fa246ffd0e1b22ecd70a50575bd5775d
297e649a3bfcaa89f9609bb00ca2e73db992b5e5b5773584fb0afda4d719
ebcecb43375ac8e8ad25cdcae636dc870036bf5bfd56b2354dbf6c1b4362
2b622e21a01e1c3b2521973e1ee10093b66b73ac25a4c7d9bf5f1d1a7a6
e5b3e229300dd474fcb80ba03712b25

Verification (original message): True

Verification (modified message): False